

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

ASSESSMENT TOOL 2 – TECHNICAL PRESENTATION

	CSA0315	Course Title	Data Structures
CO Assessed	CO1	Assessment	Assessment Tool 2: TECHNICAL PRESENTATION
Weightage	35% of CIA	Total Marks	100
CO4 Statement	<i>Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL2, BL3)</i>		
Student Name	K SAMUEL RAJ	Reg. No.	1913260140
Section/Batch	Slot D	Date	06-10-2026

Code of Conduct:

I, K SAMUEL RAJ, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

SAMUEL the student:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Technical content accuracy	30%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts	
Problem identification	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established	

Use of tools	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure	
Communication	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence	
Professionalism	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand	



Choosing a Data Structure for CPU Scheduling

Ready queue in a time-sharing operating system: time and space efficiency

The problem: many processes, one CPU

Fair turns

Each process gets a time slice (typically 10 to 100 ms)

Constant churn

Processes arrive, block, wake up and finish all the time

Hot path

The scheduler runs on every timer tick and context switch

Scarce memory

Kernel memory is limited, so overhead must stay small

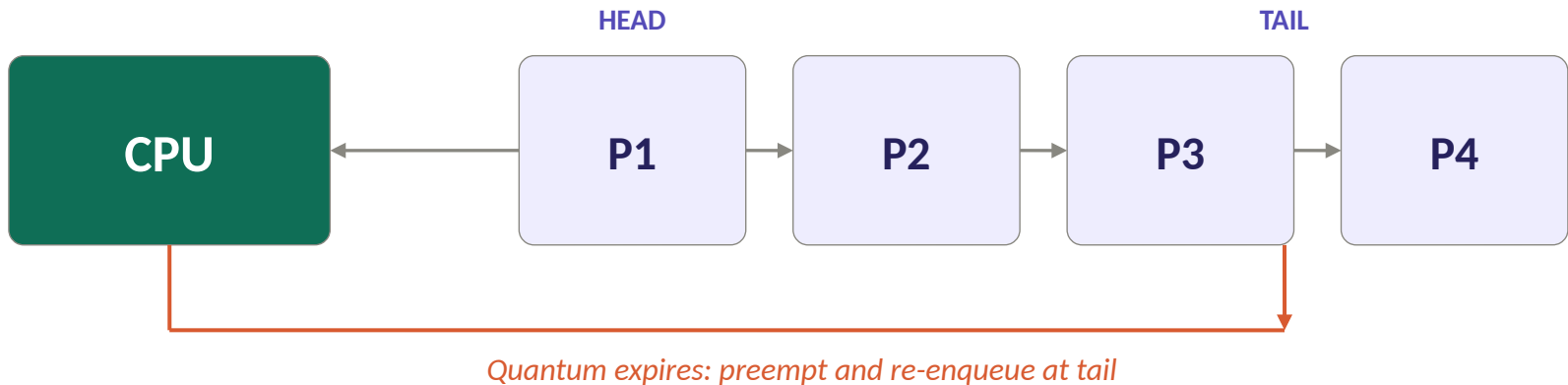
Key need: fast insert at the tail and fast removal from the head

Candidate structures compared

Structure	Enqueue	Pick next	Space	Verdict
Array (shift on remove)	$O(1)$	$O(n)$	$O(n)$	Slow dequeue
Sorted linked list	$O(n)$	$O(1)$	$O(n)$	Slow insert
Circular queue	$O(1)$	$O(1)$	$O(n)$	Best for Round Robin
Binary heap	$O(\log n)$	$O(\log n)$	$O(n)$	Best for priority
Red-black tree	$O(\log n)$	$O(1)^*$	$O(n)$	Weighted fairness

* Leftmost node is cached, so choosing the next process is constant time.

Recommendation: a circular FIFO queue



1 Dispatch

Remove the head process in $O(1)$

2 Run

Execute for one time quantum

3 Re-queue

Append to tail in $O(1)$, or drop if finished

Time efficiency: constant cost per decision

$O(1)$

Enqueue at tail

$O(1)$

Dequeue from head

$O(1)$

Pick next process

Why it matters

- The scheduler runs on every timer tick, so its cost is paid thousands of times per second.
- With 1,000 ready processes, an array shift moves about 1,000 entries per switch; the queue moves none.
- A heap needs about 10 steps ($\log n$) for the same job, which only pays off when priorities matter.
- Predictable $O(1)$ cost keeps context-switch latency steady under load.

Space efficiency: $O(n)$ total, $O(1)$ extra

Circular array

- Fixed buffer, no pointers
- Contiguous, cache-friendly
- Capacity must be chosen up front

Linked via the PCB

- Next pointer lives in the process control block
- No separate allocation per node
- Grows and shrinks with the process count

The queue adds only a head and a tail pointer, so extra space stays $O(1)$

Conclusion: match the structure to the policy

Time-sharing with equal turns: use a circular FIFO queue

$O(1)$ enqueue and dequeue, $O(n)$ total space, $O(1)$ extra space, simple and fair.

If priorities matter

Use a binary heap or multilevel feedback queues:
 $O(\log n)$ selection, or one $O(1)$ queue per level.

If weighted fairness matters

Use a red-black tree, as in Linux CFS: $O(\log n)$ insert, cached leftmost for the next pick.